新型后门睡眠技术赏析

Z2O安全攻防 2023年8月19日 21:06 北京

以下文章来源于M01N Team ，作者天元实验室

背景

睡眠技术是恶意载荷为达到间歇性执行、隐藏自身活动以逃避内存扫描的目的所采用的隐匿策略，在终端对抗中占有重要的一席之地。高隐匿性的睡眠方案能够在沙箱对抗中有效地保护样本的恶意行为不暴露，也帮助载荷在越来越先进的内存扫描等终端对抗方案中匿迹隐形。

随着Nighthawk、BruteRatelC4等新兴C2开始逐渐走进攻击者视野，在针对Cobalt Strike和Metasploit Framework这种主流C2控制框架的检测技术越来越成熟的今天，这些新兴C2的后门工具搭载着更加先进的技术，对EDR等终端检测解决方案发起挑战。**本文将对Nighthawk、BruteRatelC4后门中出现的独特的睡眠技巧进行介绍。**

01睡眠加密的暴露风险

为了躲避内存扫描，有效载荷在绝大部分时间里往往处于睡眠状态，并在这段时间里加密自身所处内存，缩短载荷本体暴露在内存中的窗口期。在Cobalt Strike等传统C2中，往往利用一段独立的混淆保护逻辑(Sleep Stub)实现对内存空间的加密，但Sleep Stub中对载荷的加密逻辑本身既存在于一段可执行空间中，又无法对该空间进行加密，只能暴露在内存空间中，因此Sleep Stub实际上在保护执行载荷的同时也引入了新的检测特征。尽管Cobalt Strike的开发商提供了允许用户自定义Stub加解密逻辑，但这依然无法解决“我加密我自己”的逻辑悖论问题。

此外，针对常见的睡眠手段，如利用Sleep()、CPU时间戳等实现的睡眠行为已经有了非常成熟的对抗方式，通过线程状态筛查、栈回溯、敏感API监控等方式能够较好地对潜在的恶意载荷进行筛查。这些睡眠方式已无法很好地保护载荷不被发现，需要新的睡眠技术来进一步增强在内存中的对抗能力。

为了解决内存特征暴露和睡眠方式被监控的这两个问题，Nighthawk等红队工具在睡眠调度机制和睡眠方式上引入了新的解决方案。

02 新型睡眠调度机制

2.1 利用线程池机制实现自身加密

在去年出现的Ekko[1]项目中，利用线程池的API：CreateTimerQueueTimer创建计时器，在回调中调用内存权限修改、内存加解密与延迟执行的功能代码。Ekko将CreateTimerQueueTimer的回调函数设置为NtContinue，将当前线程的上下文修改为ThreadContext参数的内容，从而指定执行流程与执行参数。

借助线程池相关机制，使得代码与数据均可对自身进行加密，实现更加彻底的内存加密混淆。作为C2领域悄然兴起的商业攻击模拟工具Nighthawk，已然将类似技术进行了集成，以发挥更加强大的检测逃逸能力。

2.2 利用APC队列机制实现自身加密

该利用方式与APC注入类似，利用了能在特定线程环境中异步执行的异步过程调用机制：通过构造特定的调用上下文，并将其作为回调插入到APC队列。与Ekko项目一样，借助NtContinue这一非公开API将当前线程上下文修改为ThreadContext参数所指定的内容，从而实现对执行流程的控制。如下面的代码片段所示，之所以称之为异步过程调用，就是因为其调用上下文的设置与执行是分开的。通过异步调用，可以由程序先设定调用的目标地址与参数，将APC请求挂入队列，然后等待操作系统调度执行即可，由此即可实现对当前进程空间的加解密。

```
QueueUserAPC(NtContinue, NtCurrentThread(), (DWORD64)&RopProtRW);//VirtualProt
QueueUserAPC(NtContinue, NtCurrentThread(), (DWORD64)&RopMemEnc);//加密
QueueUserAPC(NtContinue, NtCurrentThread(), (DWORD64)&RopDelay);//WaitForSingl
QueueUserAPC(NtContinue, NtCurrentThread(), (DWORD64)&RopMemEnc);//解密
QueueUserAPC(NtContinue, NtCurrentThread(), (DWORD64)&RopProtRX);//VirtualProt
```

利用APC队列机制实现睡眠的方式与Ekko类似，调用的API是QueueUserAPC。QueueUserAPC的调用链关系为：

```
QueueUserAPC->QueueUserAPC2->syscall:ZwQueueApcThreadEx2->KiUserApcDispatcher-
```

这种睡眠方式与Ekko一样，能够完全对自身内存进行加密，很大程度上解决了“我无法加密我自己”这一问题。在此基础上，如何实现更加隐蔽、更加难以被检测的睡眠方式，进一步推动了攻防的博弈。

03 新型睡眠方式

有效载荷利用命令控制阶段间隔间的休眠对自身进行加密等保护以躲避防护产品对内存的扫描，规避自身暴露。可以说，睡眠技术是C2工具内存扫描对抗的基础。在Nighthawk和BruteRatelC4中，作者采用了若干种不同的等效休眠策略，能够在不调用Sleep()的前提下达到同等的延迟执行效果。

3.1 延迟执行

利用的函数原型如下：

函数声明	函数功能

NtDelayExecution(BOOL Alertable, PLARGE_INTEGER DelayInterval)	在指定时间内暂停线程

NtDelayExecution作为Sleep的替代方案，在指定时间内暂停线程，达到与Sleep一样的效果，常以Syscall的方式被攻击者调用。

3.2 利用等待信号量等对象机制

信号量机制是用于实现进程互斥、进程同步的操作系统机制。在互斥访问中，存在互斥等待的情况，并且操作系统提供了超时机制以避免死锁的发生。可设置超时时间的等待，这正好符合了定时休眠的需求，因此一系列与等待信号量等对象机制相关的API也被滥用于实现休眠。可被滥用的函数说明如下：

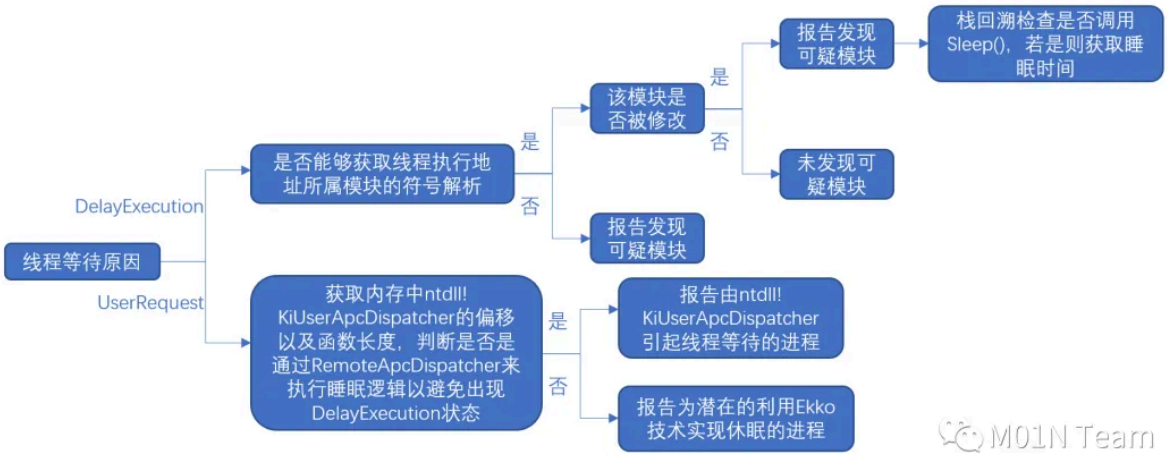
函数声明	函数功能
WINBASEAPI DWORD WINAPI WaitForSingleObject(HANDLE hHandle, DWORD dwMilliseconds);	在超时前挂起线程等待信号量
NTSTATUS NtWaitForSingleObject(HANDLE Handle, BOOLEAN Alertable, PLARGE_INTEGER Timeout);	在超时前挂起线程等待信号量
NTSTATUS NtWaitForMultipleObjects(ULONG ObjectCount, const HANDLE* ObjectsArray, BOOLEAN WaitAll, BOOLEAN Alertable, PLARGE_INTEGER TimeOut);	在超时前等待多个对象处于信号状态
NTSTATUS NtSignalAndWaitForSingleObject(HANDLE ObjectToSignal, HANDLE WaitableObject, BOOLEAN Alertable, PLARGE_INTEGER Timeout);	在超时前等待指定的对象达到状态

利用等待信号量API中的超时机制来代替睡眠，用超时时间控制睡眠时间。

利用上述睡眠替代方案，结合APC队列机制，一方面减少了敏感API的使用，增加发现难度，另一方面解决了自身加密的问题，规避了由Sleep Stub导致的特征暴露。

04 致盲Hunt-Sleeping-Beacon

随着攻防技术的不断发展，新技术带来的新暴露面逐渐清晰，针对睡眠的对抗与检测技术也层出不穷。Hunt-Sleeping-Beacon项目[2]是针对睡眠载荷的检测项目，其实现原理是通过遍历进程的线程，获取线程状态，并结合线程执行所属模块信息或栈回溯结果判断处于Delay Excution和UserRequest状态下的线程是否可疑。其中，睡眠时的线程状态是一个重要的评判指标，其判断逻辑如下图所示：



可见，尽管利用延迟执行的思路代替了睡眠函数的使用，但在终端上还是会暴露一定的行为特征。使用上面介绍的睡眠方式时的线程状态以及隐匿效果如下表所示：

睡眠方式	相关API	睡眠时线程等待状态	自身加密	Hunt-Sleeping-Beacon检测结果
延迟执行	NtDelayExecution	DelayExcution	不支持	被检出可疑DelayExcution线程状态
利用等待信号量等对象机制	WaitForSingleObject、NtWaitForSingleOb	UserRequest	不支持	未检出

	ject 、 NtWaitForMultipleObjects			
利用等待信号量等对象机制	NtSignalAndWaitForSingleObject	DelayExcution	不支持	被检出可疑DelayExcution线程状态
利用APC队列调用睡眠相关API	QueueUserAPC	UserRequest	支持	被检出使用了QueueUserAPC(KiUserApcDispatcher)实现睡眠的Ekko/Nighthawk 特征

结合线程状态的Sleeping-Beacon筛查能够一定程度上识别处于可疑状态下的线程，但对于利用等待信号量对象机制的睡眠方式就无能为力了。因此，在Nighthawk、Havoc等C2均使用了信号量相关API利用延时执行实现睡眠以达到更好的内存防御规避效果。

05 总结

睡眠技术是终端对抗上非常重要的一部分，睡眠与混淆的结合大幅缩小了内存检测、内存扫描的有效窗口。在大部分的睡眠时间中，其所在的内存区域经过了内存属性修改、内存加密等混淆保护，难以被扫描规则识别。实现睡眠的方式也多种多样，各种实现方法的特征也各有不同。因此，对内存中可疑的睡眠模块的检测需要内存属性、线程状态、内存空间上下文等多方面的信息综合判断。

附录：参考链接

[1] https://github.com/Cracked5pider/Ekko/tree/main
[2] https://github.com/thefLink/Hunt-Sleeping-Beacons/tree/main